Design and Evaluation of a Latency-Deterministic FPGA-Based High-Frequency Trading System

Sachin V Nagaraddi, Vamshikrishna V Bidari
Department of Electronics & Communication Engineering
National Institute of Technology Karnataka Surathkal, India
241EC246, 241EC258
Under the guidance of Dr. Sumam David S.

***Abstract*—In high-frequency trading, temporal determinism and predictable tail latency are strict requirements that are challenging to guarantee in software-based environments due to operating system scheduling and hardware interrupts. This work presents the microarchitecture and evaluation of a fully deterministic, hardware-accelerated trading pipeline implemented on a Field Programmable Gate Array. To resolve the variable latency bottlenecks typically associated with order book memory management, we introduce a hybrid memory architecture. This structure utilizes a parallel Top-K register cache for constant-time access to the most competitive market prices, combined with a deep Block RAM storage layer governed by a sliding window pointer to eliminate linear memory shifting. Empirical measurements collected via on-chip hardware counters demonstrate an end-to-end processing latency of 7 clock cycles, or approximately 70 nanoseconds at 100 MHz, with zero cycle-to-cycle jitter. Furthermore, we implement a backpressure-aware buffering mechanism that safely absorbs burst traffic without packet loss or timing violations. Comparative benchmarks against a software baseline highlight the complete removal of context-switching jitter, validating the superiority of purely digital pipelines for latency-critical financial applications.**

***Index Terms*—Field Programmable Gate Arrays (FPGA), High-Frequency Trading (HFT), Latency Determinism, Electronic Order Book, Microarchitecture.**

I. Introduction

In electronic trading systems, market data changes rapidly, and reacting to these updates with strict temporal precision is critical. Delays of a few microseconds can result in the processing of outdated market conditions and delayed trade executions. For these applications, worst-case latency and cycle-to-cycle timing variation are more significant metrics than average throughput. Software-based trading systems executing on general-purpose processors suffer from unpredictable latency spikes. These spikes are introduced by operating system context switching, dynamic frequency scaling, and cache misses, creating an unreliable tail latency profile that is unacceptable during high-activity market events.

Field Programmable Gate Arrays (FPGAs) provide an effective platform to address these limitations [1], [2], [6]. By eliminating the operating system entirely, FPGAs offer fixed, cycle-accurate execution. However, achieving strict determinism requires custom microarchitectures, particularly for managing the electronic limit order book where sorting operations traditionally introduce variable latency.

This paper focuses on the microarchitectural design of a deterministic trading engine. The primary contributions of this work are as follows:

- A cycle-deterministic Register Transfer Level (RTL) pipeline that processes fixed-length market data and generates trading decisions in a strict, bounded timeframe.
- A backpressure-aware order book memory system that buffers burst traffic gracefully without dropping packets or stalling the critical processing path.
- A hybrid Top-K and Block RAM (BRAM) data structure that achieves constant-time operations by removing the need for physical data shifting during order cancellations.

The remainder of this paper is organized as follows. Section II reviews the background of financial market feeds and existing FPGA-based architectures. Section III details the proposed microarchitecture and system design. Section IV outlines the experimental methodology and hardware setup. Section V discusses the evaluation results, and Section VI provides limitations and future work, followed by the conclusion in Section VII.

II. Background and Related Work

A. Financial Market Feeds

Modern financial exchanges transmit real-time market data to participants using highly optimized multicast protocols. A prominent example is the National Stock Exchange (NSE) Tick-By-Tick (MTBT) protocol, which disseminates order additions, modifications, and cancellations using a sequenced, variable-length packet structure [3]. To achieve strict temporal determinism in a hardware environment, parsing variable-length packets introduces variable cycle costs. Consequently, this architecture utilizes a simplified, deterministic 12-byte binary protocol. This fixed-length format preserves the core operational semantics of real-world feeds, specifically order addition, cancellation, and execution, while guaranteeing a constant-time parsing overhead suitable for Field Programmable Gate Array (FPGA) processing.

B. Existing FPGA Architectures

Traditional hardware-accelerated order books often map software data structures directly into FPGA block memory. Existing literature and reference implementations, such as standard sorted-array or pure-BRAM designs, typically utilize

linked lists to maintain price-time priority [7], [8]. While these implementations achieve high average throughput, they suffer from data-dependent latency. Inserting or deleting an order in a heavily populated BRAM array requires $O(N)$ memory shifting operations. As the size of the order book grows, the number of clock cycles required to update the memory increases. This variance introduces significant cycle-to-cycle jitter, violating the strict temporal determinism required for high-frequency trading.

III. Microarchitecture and System Design

A. System Constraints and Partitioning

The proposed system is designed and synthesized for the Zynq-7020 Programmable Logic (PL) fabric, specifically bypassing the processing system (ARM cores) to eliminate software interrupt latency. The microarchitecture operates at a fixed 100 MHz clock frequency, resulting in a 10 nanosecond clock period. The environment is constrained to a 4-stock universe. To isolate the hardware evaluation from complex Ethernet MAC overheads, data is ingested via a custom 115200 Baud Universal Asynchronous Receiver-Transmitter (UART) interface [4], [5]. The Host PC acts as a market data generator and features a custom real-time Python graphical dashboard. This dashboard captures the returning UART stream to continuously visualize the live order book state, calculate moving averages, and display the peak cycle-jitter metrics for both the hardware and software pipelines.

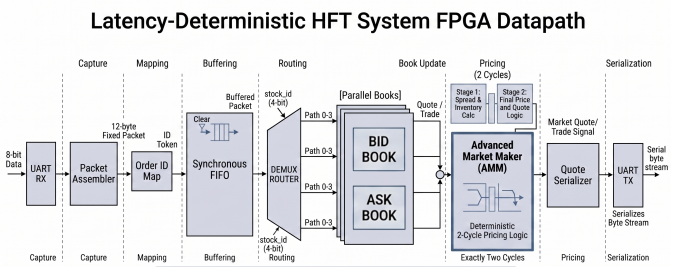

Fig. 1. Top-level System Architecture of HFT Datapath, illustrating the flow from the physical layer interface through packet parsing, order book management, and trading logic.

B. Packet Parsing and Ingress

The ingress stage consists of a packet assembler and a finite state machine (FSM) parser. The assembler collects the serial byte stream into a 96-bit register. Upon detecting a valid Start-of-Frame and End-of-Frame sequence, the parser FSM extracts the discrete fields. Because the packet length is fixed at 12 bytes, the parser guarantees a deterministic cycle count for every message.

To absorb burst traffic, the system employs a backpressure-aware buffering mechanism. A central 80-bit synchronous FIFO stores the parsed order data. In parallel, a 32-bit timestamp FIFO records the exact value of a free-running global timer at the moment the packet parsing completes. This allows the system to calculate precise queue wait times and

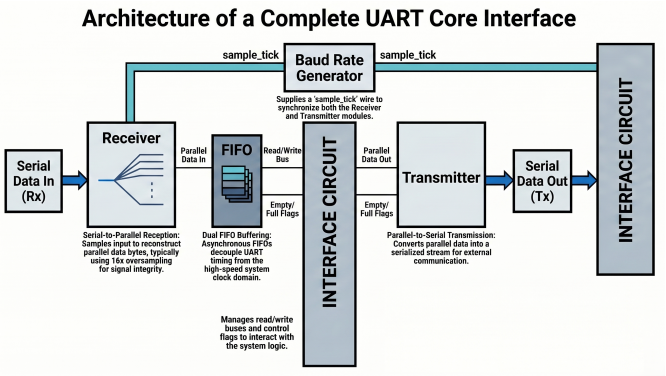

Fig. 2. Physical layer interface utilizing custom UART core with input FIFOs to ensure cycle-accurate data ingestion.

TABLE I
Deterministic 12-Byte Hardware Packet Structure

Byte Index	Field Name	Size	Description / Mapped Values
0	SOF	1	Always `0xAA` for frame alignment.
1	MsgType	1	`0x4E` ('N' Add), `0x58` ('X' Cancel), `0x54` ('T' Trade), or `0x4F` ('O' Quote).
2–3	Token / Latency	2	Inbound: 16-bit Stock Token ID. Outbound: 16-bit hardware latency cycle count.
4	Side	1	`0x42` ('B' Buy) or `0x53` ('S' Sell).
5–7	Price	3	24-bit fixed-point integer price representation.
8–9	Quantity	2	16-bit integer representing the number of shares.
10	Flags / Stock ID	1	Upper 4 bits contain the routing `stock_id`. Lower 4 bits are reserved flags.
11	EOF	1	Always `0x55` to safely terminate the packet.

empirically prove backpressure resilience during market data bursts.

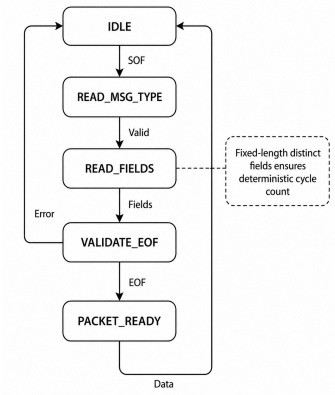

Fig. 3. State transition diagram of the packet parser, highlighting the fixed-length field extraction sequence.

Following the parser, the extracted 16-bit stock token is passed through a Global Order ID Map. This single-cycle pipeline stage enriches the packet by mapping the external exchange token to a dense 4-bit internal `stock_id`. This internal identifier directly drives the downstream demulti-

plexer, efficiently routing the order to the correct parallel stock pipeline without requiring complex associative lookups in the critical path.

C. Hybrid Order Book Engine

The core novelty of this microarchitecture is the hybrid memory structure, which guarantees that all order book operations execute in fixed clock cycles independent of the data distribution. The architecture separates the bid and ask sides into distinct, parallel processing blocks.

Each block contains a Top-K Cache implemented using parallel hardware registers rather than RAM. This cache stores the most competitive market prices. When a new order arrives, parallel comparators simultaneously evaluate the incoming price against all K registers. If the price is highly competitive, the cache updates its internal registers in a single clock cycle, achieving $O(1)$ constant-time access.

Orders falling outside the Top-K threshold spill over into a deep storage module implemented as a Circular Sparse Block RAM (BRAM). To eliminate the $O(N)$ physical shifting penalty traditionally associated with BRAM arrays, the deep storage utilizes a logical sliding window architecture with a depth of 512 levels. Rather than moving underlying memory contents during market price drifts, the hardware maps incoming prices to physical BRAM addresses using circular pointer arithmetic. When the market price window shifts, the control Finite State Machine (FSM) simply enters a `SHIFT_UP` or `SHIFT_DOWN` state to increment or decrement the `base_ptr` register in a single clock cycle. This transforms a variable-latency memory shift into a strict $O(1)$ deterministic operation.

Furthermore, the BRAM maintains a sparse representation of the order book by pairing the stored quantities with a 512-bit `price_mask` register that tracks active price levels. To facilitate immediate $O(1)$ refills to the Top-K cache when orders are canceled or executed, the system employs a dual-port architecture. Port A handles FSM-driven read-modify-write updates utilizing the strict state sequence of `IDLE`, `MODIFY_READ`, `MODIFY_WAIT`, and `MODIFY_WRITE`.

Simultaneously, Port B utilizes a highly optimized 3-stage pipelined datapath to supply the cache. In the first stage, 24-bit index boundary arithmetic is explicitly isolated within dedicated Digital Signal Processing (DSP) slices to guarantee static timing closure at 100 MHz, clamping the result to a safe 9-bit limit. The second stage applies these calculated boundaries to dynamically generate a 512-bit active mask. In the third stage, this mask is fed into a 16×32 Hierarchical Priority Encoder, which instantly identifies the highest-priority active 9-bit index, calculates the physical read address by adding the `base_ptr`, and supplies the next best price to the Top-K cache entirely in the background. This ensures the critical trading path is never interrupted.

D. Advanced Market Maker (AMM) and Trading Logic

The trading logic module advances beyond simple threshold triggers by implementing a fully pipelined Advanced Market Maker (AMM). Operating in exactly two clock cycles using

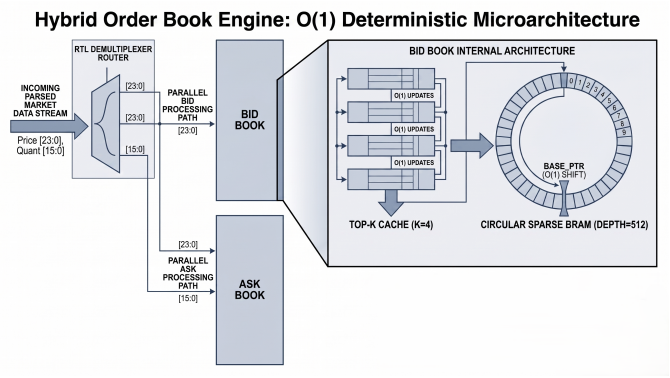

Fig. 4. Hybrid order book architecture demonstrating the parallel split between the Top-K register cache and the deep storage BRAM.

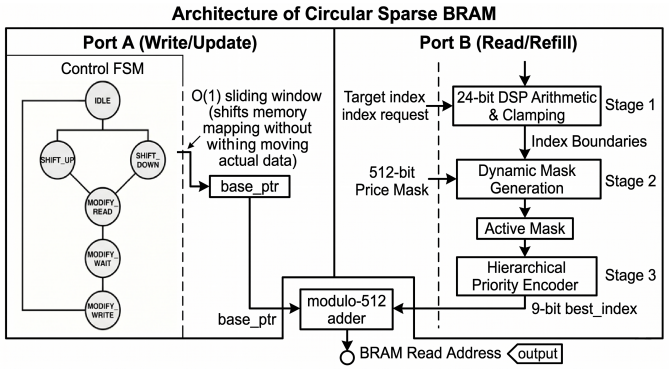

Fig. 5. Internal microarchitecture and dataflow of the Circular Sparse BRAM, illustrating the read-modify-write FSM, hierarchical price masking, and sliding-window pointer arithmetic for $O(1)$ latency deterministic access.

dedicated Digital Signal Processing (DSP) slices, the AMM calculates dynamic quotes based on the order book state and internal inventory. In the first cycle, the module computes the mid-price and market spread, while simultaneously accumulating the executed fill quantity to determine the current inventory position. An inventory skew factor is derived via an arithmetic right shift. In the second cycle, parallel DSP Arithmetic Logic Units (ALUs) apply the margin and inventory skew to the mid-price to generate the final outbound bid and ask quotes. Additionally, a hardware risk-gate continuously monitors the inventory, automatically disabling outbound quotes if the accumulated position breaches the predefined `MAX_POS` limits.

E. Formal Latency Definition and Pipeline Timing

System latency is empirically defined as the number of hardware clock cycles elapsed between the assertion of the `parse_valid` signal (marking the completion of packet ingestion) and the completion of the trading decision generation after the advanced market maker functional unit.

The critical path through the microarchitecture is strictly bounded. Under normal operating conditions where the central FIFO is not congested, the processing latency is derived from the following pipeline stages: 1 cycle for Global Order ID Map enrichment and data demultiplexing, 1 cycle for the Top-

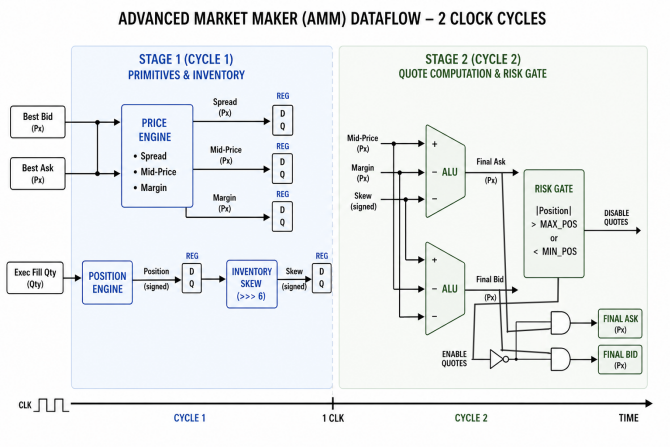

Fig. 6. Deterministic trading logic pipeline showing the flow from order book update to trade decision generation, and position-based risk gating.

K cache parallel evaluation, 3 cycles for the Circular Sparse BRAM update and spillover, and 2 cycles for the Advanced Market Maker (AMM) computation in the DSP slices. This yields a fixed, data-independent latency of exactly 7 clock cycles. At the target frequency of 100 MHz, the system achieves a highly deterministic 70 nanosecond tick-to-trade latency.

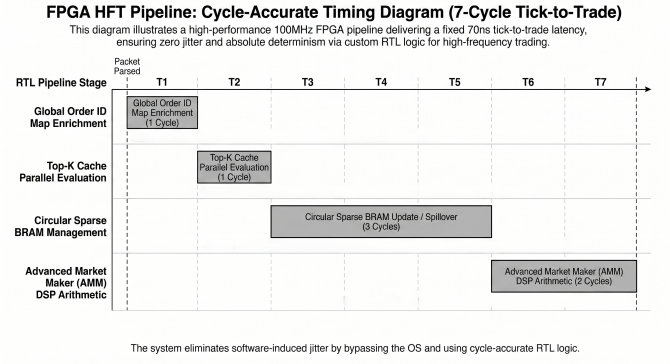

Fig. 7. Cycle-level pipeline timing diagram illustrating the deterministic 7-cycle dataflow from packet ingress (`parse_valid`) to the completion of the trading decision generation after the advanced market maker functional unit.

IV. Experimental Setup and Methodology

A. Hardware Setup and Empirical Latency Definition

The hardware pipeline is synthesized and deployed on a Digilent ZedBoard featuring the Xilinx Zynq-7020 System-on-Chip (SoC). To ensure absolute execution isolation, the ARM Cortex-A9 Processing System is bypassed, and the trading pipeline runs exclusively within the Programmable Logic fabric. Data is ingested and transmitted via a custom Universal Asynchronous Receiver-Transmitter (UART) interface operating at 115200 baud.

To evaluate temporal determinism, a 32-bit hardware cycle counter is embedded directly within the data path. Latency is empirically defined as the exact number of 100 MHz clock cycles elapsed between the assertion of the `parse_valid` signal (indicating a fully received and parsed packet) and the completion of the trading decision generation after the advanced market maker functional unit. This on-chip measurement prevents external transmission delays from corrupting the core logic evaluation.

B. Software Baseline and Testbench

A Python-based testbench running on a Host PC serves as the market data generator. It streams fixed-length packets to the FPGA under three distinct traffic profiles: a Normal mode for functional verification, a Deterministic mode (evenly spaced packets) for baseline latency extraction, and a Burst mode (rapid sequential injections) for backpressure stress testing.

To provide a rigorous architectural baseline, a software equivalent of the trading engine was implemented in C++ and executed on an Intel Core Ultra 5 225H processor. The software baseline implements the limit order book using standard `std::map` (red-black tree) data structures to maintain price-time priority. This highlights the fundamental algorithmic bottleneck of software processing: the time complexity for processing a single order packet is $O(\log M + \log L)$, where M is the number of active order tokens and L is the number of active price levels. While the software inherently requires this logarithmic traversal latency for insertions and deletions, the hybrid FPGA hardware flattens these structures into memory arrays and registers, reducing the computational bounds to a strictly deterministic $O(1)$ latency.

For performance measurement, the software implementation utilizes the `<chrono>` library high-resolution clock to measure execution time in nanoseconds. For a direct graphical comparison against the hardware, the software nanosecond latency is mathematically normalized to equivalent 10 nanosecond FPGA clock cycles. Ultimately, this baseline highlights the severe latency variations introduced by a combination of logarithmic tree traversals, operating system context switching, thread migration across Performance and Efficiency cores, and dynamic frequency scaling.

V. Results and Discussion

A. Resource Utilization and Timing

The proposed microarchitecture was synthesized and implemented using Xilinx Vivado for the Zynq-7020 device. To demonstrate the hardware scalability of the parallel order book routing architecture, Table II compares the resource footprint of a single-stock pipeline against the fully scaled 4-stock universe.

TABLE II
Device Resource Utilization (Zynq-7020)

Resource Type	1-Stock Util (%)	4-Stock Util (%)	Available
Look-Up Tables (LUT)	16.11 %	54.93 %	53,200
Flip-Flops (Registers)	5.32 %	13.27 %	106,400
Block RAM (BRAM)	59.64 %	61.43 %	140
DSP Slices	1.82 %	7.27 %	220

The design successfully met static timing closure at 100 MHz with a Worst Negative Slack (WNS) of 0.038 ns. The estimated total on-chip power consumption is 0.697 W, validating the efficiency of the microarchitecture for edge-deployed financial accelerators.

B. Empirical Latency and Determinism Analysis

The primary objective of this architecture is the elimination of cycle-to-cycle latency variance. Figure 8 illustrates the latency distribution of the FPGA hardware pipeline compared against the C++ software baseline.

The software implementation exhibits a wide distribution tail. While the CPU operates at a significantly higher base clock frequency, operating system jitter and cache misses cause execution times to fluctuate unpredictably into the microsecond range. Conversely, the empirical hardware measurements confirm that the FPGA processes every Top-K cache hit in exactly 7 clock cycles. At a 100 MHz operating frequency, this translates to a strictly bounded 70 nanosecond tick-to-trade latency with zero jitter.

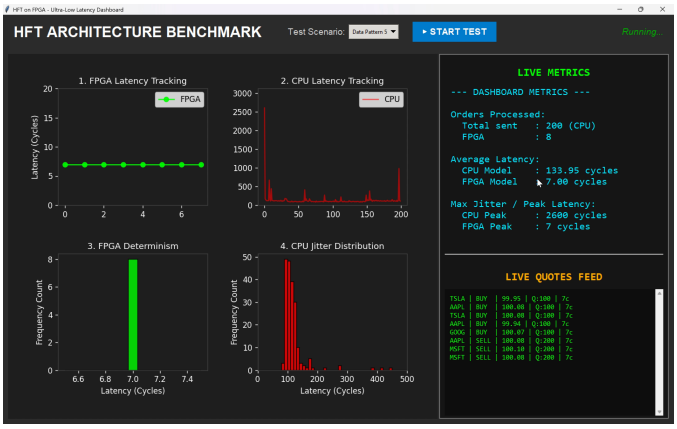

Fig. 8. Latency distribution comparison. The FPGA executes in a deterministic 70 ns fixed-latency regime, while the software baseline exhibits a wide variance tail caused by operating system jitter.

C. Workload and Scalability Sanity

The routing layer successfully demultiplexes incoming traffic across the four parallel stock pipelines without contention. The deterministic 70 nanosecond latency is preserved uniformly across all simulated market symbols and equally across both Bid and Ask workloads, proving the data-independence of the microarchitecture.

VI. Limitations and Future Work

While the proposed microarchitecture guarantees strict temporal determinism within the processing logic, the system scope presents certain boundaries. First, the data ingestion relies on a custom UART interface, which possesses insufficient bandwidth for real-world high-frequency trading compared to PCI Express (PCIe) or 10-Gigabit Ethernet (10GbE) links. Second, the system processes a simplified 12-byte binary protocol rather than full exchange formats like the Financial Information eXchange (FIX) or FIX Adapted for Streaming (FAST) protocols. Finally, the current parallel hardware routing limits the system to a 4-stock universe.

Future work will address these constraints by migrating the physical layer to a 10GbE Media Access Control (MAC) core and expanding the routing logic to support a larger universe of market symbols, utilizing deeper BRAM configurations to accommodate complete exchange symbol dictionaries.

VII. Conclusion

This paper presented the design and evaluation of a fully deterministic, hardware-accelerated high-frequency trading pipeline. By replacing variable-latency software arrays with a hybrid Top-K register cache and a sliding-window Block RAM architecture, the $O(N)$ latency penalty of traditional memory shifting was eliminated. The empirical results demonstrate that the FPGA microarchitecture successfully evaluates market data and generates trading decisions in a constant, data-independent 70 nanoseconds. The system proved highly resilient to burst traffic, maintaining structural integrity and latency boundaries under stress. Ultimately, this work validates that pure Register Transfer Level designs offer the absolute temporal determinism required to eliminate the unpredictable execution jitter inherent in modern software trading systems.

References

[1] Endrias, Tony, and Natnael, "An HFT (High Frequency Trading) Accelerator," Massachusetts Institute of Technology (MIT), 6.111 Final Project Report, Fall 2019. [Online]. Available: https://web.mit.edu/6.111/volume2/www/f2019/projects/endrias_Project_Final_Report3.pdf

[2] C. Leber, B. Geib, and H. Litz, "High frequency trading acceleration using FPGAs," in *2011 21st International Conference on Field Programmable Logic and Applications*, Chania, Greece, 2011, pp. 317-322.

[3] National Stock Exchange of India (NSE), "Real-time Capital Market / Currency Derivatives Tick-By-Tick (MTBT) Protocol," Version 6.4.

[4] D. J. Marion (FPGA Discovery), "UART in Verilog on Basys3 FPGA using PuTTY," YouTube, 2021. [Online]. Available: https://youtu.be/L1D5rBwGTwY

[5] D. J. Marion (FPGADude), "Digital-Design/FPGA Projects/UART," GitHub. [Online]. Available: https://github.com/FPGADude/Digital-Design

[6] S. Nair, "FPGA Acceleration in HFT: Architecture and Implementation," Medium. [Online]. Available: https://medium.com/@shailamie/fpga-acceleration-in-hft-architecture-and-implementation-68adab59f7af

[7] Kodoh, "Orderbook: Open source High-Frequency Trading Order Book with FPGA Acceleration," GitHub. [Online]. Available: https://github.com/Kodoh/Orderbook

[8] "Building Low-Latency Order Books with Hybrid Binary-Linear Search Data Structures on FPGAs," RnD Showcase, IIITH.

[9] C. Felton, "A Fixed-Point Introduction by Example," *DSPRelated*. [Online]. Available: https://www.dsprelated.com/showarticle/139.php